

LEAP: Light Weighting Neural Network Inference through Proactive Early Exiting Prediction

Yingtao Shen¹, Xiangjie Li^{1,2}, Yehan Ma¹, Weidong Cao³, Jie Zhao⁴, An Zou¹

¹Shanghai Jiao Tong University, ²The University of Tokyo, ³George Washington University, ⁴Microsoft Corporation

Abstract—In recent years, the incorporation of early exit layers into deep neural networks has allowed inference to terminate earlier while maintaining accuracy. However, the passive decision-making involved in these static exit placement creates a dilemma: fine-grained placement may cause high performance and energy overhead due to frequent exit layer execution, while coarse-grained placement may miss early exit opportunities. Moreover, common energy-saving techniques like adjusting processor configurations are not applicable once inference begins. To overcome these challenges and improve computation and energy efficiency, we propose LEAP, a software-hardware co-design approach. On the software side, LEAP proactively predicts exit points at runtime, reducing computation by enabling early exits without requiring every pre-placed exit layer to be executed. On the hardware side, LEAP adjusts processor settings—such as frequency and voltage—based on single or multiple predicted exits to optimize energy consumption while adhering to latency requirements. Extensive experimental results show that LEAP significantly improves efficiency. Compared to standard inference, LEAP reduces computation by up to 76.4% and saves up to 83.2% in energy. Compared to state-of-the-art early exit methods, LEAP achieves up to 27.9% less computation and 57.1% more energy savings, while maintaining similar accuracy and latency.

Index Terms—Neural Networks, Early Exit, Energy Efficiency, Inference Time, Dynamic Voltage Frequency Scaling

I. INTRODUCTION

IN recent years, deep learning techniques have made remarkable advancements in various applications, profoundly influencing numerous aspects of daily life. However, these models demand substantial computational and energy resources, resulting in an increased carbon footprint from the supporting hardware, which negatively impacts the environment [1; 2]. This issue also presents challenges for deploying these models in energy-efficient yet resource-constrained systems like edge-computing hardware [3]. Researchers have noted that the increased complexity of these models mainly benefits the accurate classification of a select few complex inputs, while proving to be excessive for simpler inputs [4; 5]. Motivated by this observation, early exit strategies integrate additional intermediate exit layers [6] into specific network layers. When data is processed through the network layers, it is sequentially evaluated at each pre-placed exit layer for potential early termination opportunities, thus avoiding the need to execute subsequent layers. These auxiliary exit layers enable a significant number of high-confidence inferences to exit early, consequently improving overall efficiency.

While early exit strategies [7] have demonstrated their ability to reduce computation and energy cost during inference, they still face the challenge of determining the appropriate granularity and placement of exit layers. Fine-grained exit

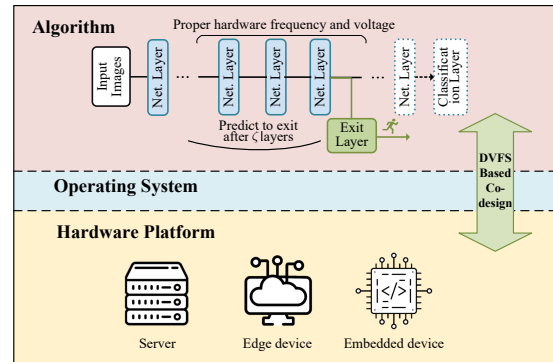


Fig. 1: The proposed software and hardware co-design: LEAP.

points can lead to significant energy overheads due to frequent execution of exit layers [8], while coarse-grained exit points could result in missed opportunities to exit. Furthermore, current early exit methods are sub-optimal from a hardware perspective, as factors such as hardware configuration (e.g., voltage) play a vital role. This constraint prevents them from fully leveraging the benefits of both sophisticated software algorithms and state-of-the-art hardware computing systems. Existing hardware utilization of early exits can only *statically* reduce computing voltage and frequency at the beginning of an inference or after triggers of an exit layer. As a result, run-time computing configuration adjustments, such as dynamic voltage and frequency scaling (DVFS), cannot be applied *dynamically* during the inference process.

To enable efficient neural network inference on resource-constrained computing platforms, we present a general software and hardware co-design framework called LEAP (Light Weighting Neural Network Inference through Proactive Early Exiting Prediction), as illustrated in Fig. 1. LEAP employs a proactive strategy to dynamically predict the most suitable exit points based on each input's features, allowing it to exploit the benefits of fine-grained exit layer placement without paying the cost of sequentially running through every exit layer, as in previous early exit methods. Moreover, LEAP predicts optimal exit points at runtime for each input while making proactive hardware control decisions by adjusting computational parameters throughout the inference process. As a result, LEAP is able to achieve superior energy efficiency compared to state-of-the-art early exit methods. Building on the LEAP framework, we implement two simple yet effective strategies oriented toward latency guarantees and optimal energy efficiency, respectively. Specifically, we first introduce a Single Predicted Exiting (SPE) principle, which performs one exit prediction and forces the network to exit at the predicted point. This approach proactively reduces computational and energy costs while ensuring network

inference latency. Secondly, we present a Multiple Predicted Exiting (MPE) principle, which generates several exit trials. This approach minimizes computation and energy costs, albeit with occasional inference deadline misses. Experimental results on CPU and GPU platforms demonstrate that both methods are effective. However, SPE is more suitable for real-time applications, whereas MPE can further save energy without introducing significant throughput delays.

In this work, we make the following contributions:

- We introduce LEAP, a comprehensive software and hardware co-design framework for optimizing neural network inference by employing proactive early exiting prediction. Specifically, it utilizes a novel early exit point prediction method to identify the optimal exit point for the network at runtime. This information is then utilized by an intelligent hardware controller to adjust computational parameters and determine when to terminate the inference process.
- Based on the LEAP framework, we develop two simple yet effective principles: Single Predicted Exiting (SPE) for guaranteed Inference Latency and Multiple Predicted Exiting (MPE) for Superior Energy Efficiency. Both principles are implemented on real CPU and SoC systems with actual power and energy measurements.
- Thorough experiments on real hardware demonstrate that LEAP outperforms state-of-the-art early exit methods in terms of computation reduction and energy savings. Our proposed approach achieves up to 76.4% reductions in computation and up to 83.2% in energy consumption in experiments with CNNs and ViTs (across various datasets) on Intel CPU and NVIDIA SoC platforms.

II. BACKGROUND AND RELATED WORK

A. Latency, Power, and Energy

In many applications, such as robots and self-driving cars, deep learning networks are released and executed periodically. The time interval between the release times of two consecutive inferences is called the inference period T . The actual execution time of each inference is called latency l , which rarely exceeds the period T . Given a computing platform and a neural network with fixed computation, the inference latency l is inversely proportional to the operating frequency of the digital logic inside the computing platform, which is called the computing frequency f . Following the basic rules of semiconductors, a higher f indicates faster computing but requires a linearly increased processor supply voltage V .

When a task is executed, the power consumption in computing, called *active power* P_{active} , is the sum of dynamic, static, and peripheral power. When the processor is idle, the power consumption, also called *idle power* P_{idle} , mainly comprises static and peripheral power. The energy E of a processor includes both active and idle power integrated with time,

$$\begin{aligned} E &= \int_0^{T_{active}} P_{active} dt + \int_0^{T_{idle}} P_{idle} dt \\ &= \int_0^{T_{active}} (P_{dynamic} + P_{static} + P_{peripheral}) dt \\ &\quad + \int_0^{T_{idle}} (P_{static} + P_{peripheral}) dt \end{aligned} \quad (1)$$

T_{active} is executing time spent for the task, and T_{idle} is the time duration when the processor is idle, which is the time interval between finishing this inference and starting the next inference, denoted by $T_{idle} = T - T_{active}$. The *dynamic power consumption* $P_{dynamic}$ originates from the activity of logic gates inside a processor, including switching power and short-circuit power. To be simplicity, we follow the classic power model in [9] and only consider the contribution of P_{switch} :

$$P_{dynamic} \approx P_{switch} = CV^2 f, \quad (2)$$

where C is the capacitance of switched logic gates. The *static power dissipation* P_{static} originates from transistor static leakage currents (such as subthreshold leakage, gate leakage, and junction leakage) when the processor is idle and not switching, which is described by

$$P_{static} = VN_{tr}I_{leakage}. \quad (3)$$

N_{tr} is the number of logic gates, and $I_{leakage}$ is the normalized leakage current for each gate. The *peripheral power* $P_{peripheral}$ is the power consumption by auxiliary devices and peripheral circuits outside the processor, like board fans and I/O ports. Since different peripherals exhibit varying power behaviors, we treat $P_{peripheral}$ as an empirically measured value that reflects the aggregate consumption of all peripheral components.

B. Early Exit with Dynamic Voltage Frequency Scaling

DVFS is a power management technique in computer architecture whereby f and V of a microprocessor can be automatically adjusted on-the-fly depending on the actual needs to conserve power and energy cost of the computing platform. Given the same inference period T , combining early exit and DVFS could effectively reduce the cost of computation resources for a deep learning network.

Originally, the deep learning networks are executed by running all network layers at the default high frequency and voltage without early exit and DVFS. We assume the inference latency (execution time) l is equal to inference period T . In this case, the energy consumption within T is $E = \int_0^T (CV_{high}^2 f_{high} + V_{high} N_{tr} I_{leakage} + P_{peripheral}) dt$. As the deployment of early exit, network layers located ahead of the early exit run at the default high frequency and voltage. After the network exits at T_{exit} , computing platform can reduce the voltage and frequency to the lowest level by DVFS until time reaches T [10]. Therefore, the energy consumption can be described by $E = \int_0^{T_{exit}} (CV_{high}^2 f_{high} + V_{high} N_{tr} I_{leakage} + P_{peripheral}) dt + \int_{T_{exit}}^T (V_{low} N_{tr} I_{leakage} + P_{peripheral}) dt$.

Since the processor voltage-frequency pair setting is approximately cubic to $P_{dynamic}$ but linear to P_{static} and latency l according to Eqs. (2) and (3), it is highly demanded to further reduce the cost of $P_{dynamic}$. An intuition to achieve this is to adjust the voltage and frequency to proper middle-level and run the network until it exits at time T . However, how to predict the exit timing and adjust the voltage-frequency pair during the inference is not trivial. In this work, we propose a solution to adjust the voltage and frequency to the proper “middle-level” at run-time based on the exit prediction. The energy cost is reduced by utilizing the inference period better.

C. Related Work

Early exits have attracted tons of attention as an important branch for dynamic inference in the past few years [6]. Since it was studied in [4], many approaches have been proposed to improve the accuracy or reduce the computation cost of exit decisions. Wang et al. [11] offered a dynamic loss-weight adjustment early-exit strategy for ResNets together with the hardware-friendly architecture of early-exit branches. To determine the placement of exit layers, Bonato et al. [12] classified the exit point for each inference object, while Panda et al. [13] and Baccarelli et al. [8] calculated the benefit and additional computation and energy cost from adding this exit layer. To further balance the accuracy, inference time, and energy tradeoffs, Wołczyk et al. [14] proposed a Zero Time Waste (ZTW) method approach by adding direct connections between exit layers and combining previous outputs in an ensemble-like manner, and Zhang et al. [15] proposed a threshold-searching-based dynamic inference mechanism through multi-classifier trained by self-distillation. Jie et al. [16] proposed a multiple Q-network based routing convolutional network to select exit for each sample adaptively. Then, to reduce the processor power and energy consumption in processing a group of network inferences with early exits, Li et al. [17] presented a feedback control approach to adjust the voltage and frequency according to the early exits, the batch workloads, and timing constraints. Nabavinejad et al. [18] introduced the design and implementation of a fast and lightweight runtime system called BatchDVFS, which searches to find the proper batch size based on the trade-off between throughput with power consumption.

Together with the proceeding of theoretical models, early exits have started being deployed in hardware and IoT systems [19]. Li et al. [20] and Zeng et al. [21] adopted device-edge synergy optimized by DNN partitioning and DNN right-sizing through the early exit for on-demand DNN collaborative inference. The strategy is feasible and effective for low-latency edge intelligence. Xu et al. [22] presented a compressed CeNN framework optimized by five different incremental quantization methods and early exit, and the FPGA implementation shows that two optimizations achieve 7.8x and 8.3x speedup, respectively, while almost no performance loss. Although the multi-exit networks in these studies can lead to a reduction in inference time and computational cost, their efficiency lies in selecting a placement of exit layers that keeps a good balance between cost-saving and inference accuracy.

It is worth noting that in recent research, a technique structurally similar to early exit has emerged in model modification: Supervised Layer-wise Training. Since the initial proposal of Layer-wise Training in [23], it has garnered increasing academic attention as a training optimization paradigm. Among them, the supervised approach typically involves inserting classifiers at intermediate layers to collect and optimize local representations. For instance, Belilovsky et al. [24] attach classifiers to each CNN layer for local representation optimization. You et al. [25] introduces auxiliary classifiers to decouple feature aggregation and transformation, significantly reducing GNN training time and memory consumption. Aly et al. [26]

enable output classifiers to incorporate both intermediate layer outputs and initial inputs, enhancing model stability through ensemble learning of all output layers. However, unlike early exit, which primarily focuses on optimizing intermediate exits for efficient inference, Layer-wise Training predominantly aims at enhancing backbone network performance. Even studies employing layer-wise classification primarily optimize gradient propagation and computation patterns rather than inference efficiency. Notably, recent trends shift toward Unsupervised Layer-wise Training without additional classifiers. For example, Karimi et al. [27] train CNNs through near-near selection based on triplet loss during forward propagation, eliminating intermediate classifiers and gradient regression. In GNN domains, based on Predictive Coding, Pina et al. [28] accelerate training by maximizing layer-wise mutual information between outputs and subsequent inputs. In summary, Early Exit preserves backbone architecture through dynamic exit layer insertion for computational efficiency, whereas Layer-wise Training modifies model structure and gradient computation to optimize backbone performance without direct inference acceleration benefits.

III. HARDWARE AND SOFTWARE CO-DESIGN: LEAP

In this section, we present an overview of our proposed framework: Light Weighting Neural Network Inference through Proactive Early Exiting Prediction. LEAP is a general framework that facilitates software and hardware co-design for neural network inference by incorporating proactive exit decision-making and hardware processor control. The general process is outlined in Algorithm 1. After the start layer L_0 , the intermediate inference result y_{L_0} will be input into this layer's exit layer. Exit layer's duty is to forecast the correct final result y' . A confidence test will be applied on this forecast. If the confidence score is larger than a pre-set threshold β , inference will exit at this layer with this expected final result y' . If not, the low-cost prediction engine (PREDICT) will accept the exit

Algorithm 1: LEAP

Input: Input data x for inference;
 Starting layer L_0 and confidence threshold β ;
 Low-cost Prediction Engine: PREDICT;
 Controller for DVFS: DVFS;
 Layerwise inference function: $f_{\mathbf{w}}(\text{Input}, \text{Layer})$;
Output: Inference result y ;
 Predicted exiting point L_{target} ;
 $L_{\text{dvfs}} = L_0$; // *Early exit start at L_0*
 DVFS.apply($V_{\text{high}}, f_{\text{high}}$); // *First run at default frequency*
for $j \leftarrow L_0$ **to** $L_{\text{total}} - 1$ **do**
 $y_{L_j} = f_{\mathbf{w}}(x, j)$; // *Do inference of this layer*
 if $j == L_{\text{dvfs}}$ **then**
 // *Activate this layer's exit layer, obtain exit result*
 $y' = g_{\mathbf{w}_{L_j}}^{(L_j)}(y_{L_j})$;
 if y' 's confidence score $> \beta$ **then**
 return $y = y', L_{\text{target}} = L_j$; // *Exit*
 else
 // *Predict next exit point if not exiting*
 $\zeta = \text{PREDICT}(y', \beta)$;
 $L_{\text{dvfs}} = L_{\text{dvfs}} + \zeta$;
 // *Control DVFS based on predicted exit point*
 $(V_{\text{mid}}, f_{\text{mid}}) = \text{DVFS.freq}(\zeta)$;
 DVFS.apply($V_{\text{mid}}, f_{\text{mid}}$);
return $y = y_{L_{\text{total}}}, L_{\text{target}} = L_{\text{total}}$;

TABLE I: A qualitative comparison between our methods (LEAP) with backbone and existing early exit methods.

	Backbone Network	Static Exit [29; 13; 8]	Edge Exit [10]	LEAP	
				SPE	MPE
Computation Cost	High	Medium	High	Medium	Low
Energy Cost	High	Medium	Medium	Low	Low
DVFS Applicability	✗	✗	✓	✓	✓
Inference Latency Guarantee	✓	✓	✓	✓	✗
Effective for Deeper Network	-	✗	✗	✗	✓
Targeted Applications	-	-	Real-time	Real-time	QoS

layer output to **predict** how much layer ζ to execute before the next exit chance. And LEAP will only activate the exit layer and prediction engine on that layer. The algorithm provides an overview and some specific operations and notations will be detailed in the following sections. Specifically, the two major components: low-cost prediction engine (PREDICT) and dynamic voltage and frequency scaling controller (DVFS) will be further discussed in Section IV and V respectively.

Given the constraints of modern computer hardware systems, it is usually impossible to achieve optimal accuracy, latency, and efficiency simultaneously, necessitating trade-offs based on use cases. Under the LEAP framework, we propose two simple yet effective strategies: SPE and MPE (Section V-A and V-B), which target guaranteed latency and low energy consumption, respectively. The single prediction strategy, SPE, forces the network to exit at the predicted point and performs DVFS based on the inference progress and deadlines. In contrast, the multiple prediction strategy, MPE, attempts exit and DVFS at the predicted point. If the exit is unsuccessful, MPE performs another round of prediction and trial. So in fact the iterative mode described in Algorithm 1 is an overview of LEAP that follows MPE strategy. Despite their simplicity, we will demonstrate through experiments on real CPU and GPU platforms that SPE and MPE effectively fulfill their design goals. In practice, one may further implement more application-specific LEAP strategies (e.g., with more complex exit and DVFS control logic) to achieve the desired trade-offs.

Unlike existing early exit methods, LEAP can accurately predict exit points in the network and dynamically adjust the processor's voltage and frequency during runtime for each inference, resulting in significant power and energy savings. As SPE only performs one prediction, it ensures a guaranteed inference latency. Meanwhile, MPE's multiple predictions and trials lead to superior computation and energy efficiency.

Table I provides a qualitative comparison between our method, LEAP with SPE and MPE approaches, and existing Static Exit methods [29; 13; 8] and LUT-based proactive DVFS method Edge Exit [10]. LEAP saves costs not only for computation but also for energy costs due to its software-hardware co-design. While as concrete LEAP strategies, SPE and MPE are designed for real-time and Quality of Service (QoS) applications respectively.

IV. ACTIVE PREDICTION OF EARLY EXITING POINT

Targeting neural network inference with early exits, we propose an approach to predict the early exit point for deep learning networks by taking the convolutional neural network (CNN), one of the popular deep learning algorithms, as an

example. The approach combines the following schemes into one continuous spectrum, which are shown in Fig. 2.

①**Unified Exit Layer:** To release the burden of executing versatile exit layers, we make the exit layers used in the network share the same topology.

②**Fine-Grained Potential Exiting Points:** Exit layers are potentially placed after each backbone layer to fully utilize the exit opportunities. To avoid the computation overhead from running every exit layer, only the layer at the predicted exit point will be executed.

③**Low-cost Prediction Engine:** The core of the framework is the low-cost prediction engine, which forecasts the possible exit point and activates the early exit layer.

A. Unified Exit Layer

In this work, all the exit layers share the same topology. The basic function of the exit layer is to get the classification results from intermediate inference flow. Therefore, the proposed unified exit layer contains a Bag-of-Features (BoF) pooling layer and a Fully-Connected (FC) layer. Let y_i be the intermediate results at the i^{th} layer and N_c be the number of object classes in the dataset. The BoF pooling layer functions as a feature aggregation to extract exit features from y_i . In the BoF pooling, a set of feature vectors called codebook are used to describe y_i by measuring the similarity between the codebook and y_i . Since the size of codebook weight is usually larger than N_c , the FC layer further works as a classifier that adjusts the result of BoF pooling to $\mathbb{R}^{N_c}$, which estimates the final output of the exit layer. Details can be found in [29]. The function of the exit layer is denoted as

$$g_{\mathbf{W}_i}^{(i)}(y_i) = g_{\mathbf{W}_i}^{(i)}(f_{\mathbf{W}}(\mathbf{x}, i)) \in \mathbb{R}^{N_c}, \quad (4)$$

where $f_{\mathbf{W}}(\mathbf{x}, i)$ stands for the intermediate result of i^{th} layer with initial network inputs $\mathbf{x}$ and overall network parameters $\mathbf{W}$. $\mathbf{W}_i$ stands for the parameters of i^{th} exit (BoF and FC) layer. And $g_{\mathbf{W}_i}^{(i)}(y_i)$ is the estimated result of the classifier.

The parameters in exit layers need offline training. Since the exit layer functions through estimating final network outputs, the same cross-entropy loss function is chosen in training W_i .

$$\mathcal{J}(p, q) = - \sum_{i=1}^{N_c} p(i) \log(q(i)), \quad (5)$$

where $p(i)$ and $q(i)$ are the true class distribution and the predicted class distribution for each object.

During training exit layers, the training set data $\mathcal{X}$ with size N are fed into the model to optimize the exit parameter through batch gradient descent. Let L_{total} be the layer number of the network, and a target vector $r \in \mathbb{R}^{N_c}$ be the correct result. At first, the model is trained without any exit layer:

$$W' = W - \eta \cdot \nabla_{\mathbf{W}} \mathcal{J}(f_{\mathbf{W}}(\mathcal{X}, L_{total}), r), \quad (6)$$

where W' is the updated backbone parameters, and η is the learning rate. Afterward, with W of the original model fixed and equipped with exit layers, the model is trained again to optimize the parameters in the BoF pooling and FC layer,

$$W'_i = W_i - \eta \cdot \nabla_{\mathbf{W}_i} \mathcal{J}(g_{\mathbf{W}_i}^{(i)}(y_i), r), i \in [1, \dots, L_{total}]. \quad (7)$$

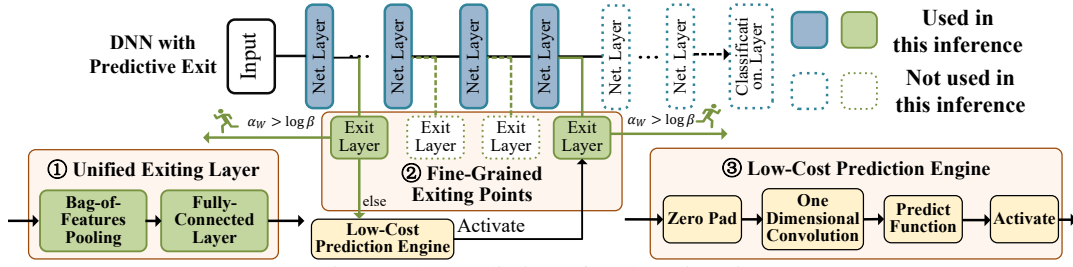


Fig. 2: The prediction of early exit point.

After training, the exit layers will output potential exit output during inference. At each exit layer, the weight ratio α_W is calculated as the maximum normalized feature weight through the LogSoftmax operation on the exit output, as shown in Eq.(8). Once the α_W is larger than $\log \beta$, the logarithm of threshold hyperparameter, the inference is terminated, and the result at this exit layer is used as the final result.

$$\alpha_W = \text{LogSoftmax} \left(\left[g_{\mathbf{W}_i}^{(i)}(y_i) \right]_k \right) = \log \left(\frac{\exp \left(\left[g_{\mathbf{W}_i}^{(i)}(y_i) \right]_k \right)}{\sum_l \exp \left(\left[g_{\mathbf{W}_i}^{(i)}(y_i) \right]_l \right)} \right), k = \arg \max g_{\mathbf{W}_i}^{(i)}(y_i) \quad (8)$$

The key idea of the exit decision is that a higher maximum feature weight indicates greater confidence in the classification. The parameter β balances accuracy and acceleration; it acts as a threshold on the maximum Softmax output of the feature weights (i.e., the highest distribution value across classification labels). A higher β leads to more accurate results, while a lower β prioritizes faster execution.

B. Fine-Grained Exiting Points

In classic early exit designs, a limited number of exit layers are placed at fixed positions and intervals, typically at $\frac{1}{3}$ to $\frac{1}{4}$ of the network depth. For example, in the model proposed in [30], two exits are located at approximately $\frac{1}{3}$ and $\frac{2}{3}$ of the network. If the first exit fails, the inference proceeds through subsequent layers until reaching the next available exit. A more practical approach, introduced in [31], places exits based on an estimated percentage of the total computational cost per layer. However, this "fixed-place" strategy overlooks the potential for exiting between two predefined exit layers. Determining where and how to place early exits remains a nontrivial challenge in balancing inference speed and accuracy.

In this work, we proposed a fine-grained exit point design, that is, placing potential exit layers on every network layer since the starting layer L_0 . L_0 is a hyperparameter specified by the user, working as an adjuster of exit overhead and potential. During the inference process, the trial of early exit begins on L_0 . If the trial succeeds, the early exit is triggered. If fails, instead of running through every exit layer from $L_0 + 1$, the position of the next layer to passing through exit layer is

$$L_{\text{target}} = L_0 + \text{PREDICT} \left(g_{\mathbf{W}_{L_0}}^{(L_0)}(y_{L_0}), \beta \right), \quad (9)$$

Where PREDICT is the function of low-cost prediction engine (will be introduced in Section IV-C) to forecast the next exit point. Specifically, our framework provide multiple prediction variant that under the circumstances that early exit does not

succeed on predicted exit layer, we will start another prediction to iteratively predict next exit layer $L_{M,i+1}$ based on the exit layer result of current layer $L_{M,i}$. That is

$$L_{M,i+1} = L_{M,i} + \text{PREDICT} \left(g_{\mathbf{W}_{L_{M,i}}}^{(L_{M,i})}(y_{L_{M,i}}), \beta \right). \quad (10)$$

The detailed design is related with hardware configuration and will be introduced in Section V-B.

C. Low-Cost Prediction Engine

1) *Prediction of Exiting Points:* The PREDICT function starts to forecast the exit point since the L_0 layer. To fully use the information from the intermediate result during the inference, the key function of PREDICT is realized by one-dimensional convolution on the intermediate estimated result $g_{\mathbf{W}_{L_0}}^{(L_0)}(y_{L_0})$ calculated from intermediate inference result y_{L_0} by exit layer at L_0 , which is summarized in Algorithm 2. To perform the convolution, we first extend the intermediate result $g_{\mathbf{W}_{L_0}}^{(L_0)}(y_{L_0})$ from $\mathbb{R}^{N_c}$ to $\mathbb{R}^{K+N_c-1}$ by zero padding, which allow for more space for the filter to cover the intermediate result,

$$J_{L_0} = \text{ZEROPAD}_{(K+N_c-1)} \left(g_{\mathbf{W}_{L_0}}^{(L_0)}(y_{L_0}) \right) = \begin{cases} g_{\mathbf{W}_{L_0}}^{(L_0)}(y_{L_0})[i], & \frac{K-1}{2} \leq i \leq \frac{K-3}{2} + N_c \\ 0, & \text{otherwise} \end{cases} \quad (11)$$

Afterwards, a tunable bilateral-smoothing kernel hyperparameter $h_i \in \mathbb{R}^K$ will serve as the convolution filter of current layer. Based on J_{L_0} and h_i , a new set of feature weight $G_{L_0+1}^{L_0+1} \in \mathbb{R}^{N_c}$ is generated through 1D peak emphasis convolution, which identify and enhance the classes with high confidence that may appear in the early stage of neural networks. This weight roughly estimates the results of exit layer placed at $L_0 + 1$,

$$G_{L_0+1}^{L_0+1}[i] = \sum_{k=0}^{K-1} h_i[k] \times J_{L_0}[i - \frac{k-1}{2}]. \quad (12)$$

By recursively replacing the intermediate result $g_{\mathbf{W}_{L_0}}^{(L_0)}(y_{L_0})$ in Eq. (11) with $G_{L_0+1}^{L_0+1}[i]$ and repeating the computation of Eq. (11) and Eq. (12), the predicted exit output at $L_0 + 2$ can be obtained and noted as $G_{L_0+2}^{L_0+2}$. Following above steps, the predicted exit output after L_0 can be calculated.

With the predicted results of exit layer placed at any layer after L_0 , we will have PREDICT function described as follows:

$$\text{PREDICT} \left(g_{\mathbf{W}_{L_0}}^{(L_0)}(y_{L_0}), \beta \right) = \zeta, \quad (13)$$

where $\zeta \in \mathbb{Z}$ represents the predicted exiting point after L_0 , which is the smallest number that satisfies

$$\text{LogSoftmax} \left(\left[G_{L_0+\zeta-1}^{L_0+\zeta} \right]_k \right) > \log \beta, k = \arg \max G_{L_0+\zeta}^{L_0+\zeta}, \quad (14)$$

where $\text{LogSoftmax}\left(\left[G_{L_0+\zeta-1}^{L_0+\zeta}\right]_k\right)$ indicates the predicted exiting confidence at layer $L_0 + \zeta$. Therefore, $L_0 + \zeta$ is the predicted exiting point from PREDICT. In case that ζ cannot be found, a hyperparameter τ is further introduced, that is $\zeta \in [1, \tau]$. τ should be no more than $L_{\text{total}} - L_0$ where L_{total} stands for the number of layers of the model, meaning that prediction result beyond the last layer is forbidden. If no integer in $[1, \tau]$ satisfies Eq. (14), we assume that $\zeta = \tau$. The prediction engine is simple enough to avoid adding notable computation costs to the network. Meanwhile, hyperparameters are introduced in the prediction. L_0 , β , h_i , and τ can be tuned by advanced users, which can balance the prediction accuracy and computation cost for different application scenarios.

Algorithm 2: Low-cost Prediction Engine

Input: Hyperparameter starting layer L_0 , β and τ ;
 exit layer result at L_0 : $g_{\mathbf{w}_{L_0}}^{(L_0)}(y_{L_0})$;
Output: Predicted exiting point: $L_0 + \zeta$;
// The prediction procedure:
for $L_0 + \zeta = L_0 + 1, \dots, L_{\text{total}}$ **do**
 // Zero padding from $\mathbb{R}^{N_c}$ into $\mathbb{R}^{K+N_c-1}$
 if $L_0 + \zeta = L_0 + 1$ **then**
 $J_{L_0+\zeta} = \text{ZEROPAD}\left(g_{\mathbf{w}_{L_0}}^{(L_0)}(y_{L_0})\right)$;
 else
 $J_{L_0+\zeta} = \text{ZEROPAD}\left(G_{L_0+\zeta-1}^{L_0+\zeta}(y_{L_0+\zeta-1})\right)$;
 // One dimensional convolution
 $G_{L_0+\zeta-1}^{L_0+\zeta}[i] = \sum_{k=0}^{K-1} h_i[k] \times J_{L_0+\zeta}[i - \frac{k-1}{2}]$;
 // Check prediction confidence
 if $\text{LogSoftmax}\left(\left[G_{L_0+\zeta-1}^{L_0+\zeta}\right]_k\right) > \log \beta$, $k =$
 $\arg \max G_{L_0+\zeta}^{L_0+\zeta}$ **or** $\zeta = \tau$ **or** $L_0 + \zeta = L_{\text{total}}$ **then**
 return ζ ;

D. Computation Cost of Each Module

Compared to the other state-of-the-art early exit approaches, LEAP's prediction engine introduces additional computational costs. However, by guiding the exit decision for potentially placed exit layers, the prediction engine can reduce the number of exit layers executed during inference, thus minimizing the exit overhead. Theoretically, the computation cost of prediction engine (Algorithm 2) is $O(N_c \times K \times \tau)$, a negligible value compared with backbone and exit layer. To prove this, we evaluated theoretical floating-point operations (FLOPs) and practical time costs on different processors for both the unified exit layer and prediction engine and take the average data of the backbone layer as the baseline, as presented in Table II. According to the first row, the prediction engine has almost no overhead, and for the exit layer, its FLOPs is also less than 15%. However, in practice, unlike the backbone layer, which benefits from hardware accelerations and software optimizations, the execution time of the exit layer increases significantly on existing general-purpose processors (particularly for GPU platforms), and the acceleration of exit layer requires specific hardware design [32]. Therefore, it is crucial to limit the number of exit layers that the intermediate inference results traverse.

V. HARDWARE COLLABORATION

Dynamic Voltage and Frequency Scaling (DVFS) is a hardware technique that balances power consumption and

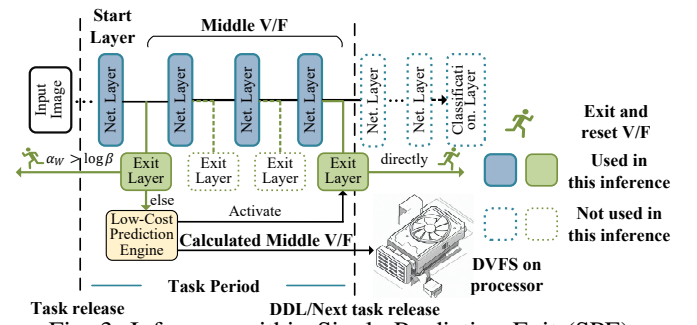


Fig. 3: Inference within Single Prediction Exit (SPE).

performance in processors by adjusting voltage and frequency based on workload. During light tasks, DVFS lowers voltage and frequency to save power. Conversely, when more computation is needed, DVFS scales up to boost performance with increased power consumption. This technology plays a crucial role in optimizing energy efficiency while preserving the essential functionality of devices, making it a cornerstone of modern computing processors. The optimal usage of DVFS requires an accurate estimation of the computation workload and setting the voltage and frequency to proper levels.

Leveraging the proactively predicted exit point, we introduce two hardware collaboration strategies: Single Prediction Exit (SPE) and Multiple Prediction Exit (MPE). These innovative approaches allow for precise adjustment of processor voltage-frequency pairs, resulting in a noteworthy reduction in energy consumption. Taking various considerations into account, the two approaches enjoy their own distinct characteristics, where the Single Prediction SPE can guarantee the inference latency while reducing the moderate energy cost, and the Multiple Prediction MPE can achieve the superior power and energy efficiency with occasional inference latency deadline misses.

A. Single Prediction with Guaranteed Inference Latency

In practice, it is impossible for the Prediction Engine's accuracy to reach 100%. Taking various considerations into account, we propose two types of predictive-exit mechanisms, each with its own distinct characteristics. The first mechanism, known as Single Prediction Exit (SPE), is illustrated in Figure 3. During an inference process, only one prediction is performed. When the inference passes through the predetermined start layer, the exit layer processes the intermediate inference result at this Start Layer. If the exit condition is not met at the Start Layer, the Prediction Engine will utilize the exit result to predict the layer to exit. Subsequently, the inference will be forced to exit directly at the predicted layer, ensuring that the total time cost of the inference meets the deadline.

During inference, once prediction is made at Start Layer, the voltage-frequency pair is adjusted to proper "middle levels" based on the predicted exiting point and run the network until it exits at time T . Given a network with L_{total} layers and a prediction engine that starts the prediction at L_0 and predicts the exiting point is $L_0 + \zeta$, the prediction engine will reduce the computing frequency (and its corresponding voltage) to

$$f_{\text{mid}} = \frac{L_0 + \zeta - L_0}{L_{\text{total}} - L_0} f_{\text{high}}, \quad (15)$$

TABLE II: Theoretical FLOPs and real system execution time for exit and prediction (taking backbone layer as baseline).

Model Module	Backbone Layer	VGG-19 Exit Layer	Prediction Engine	Backbone Layer	ResNet-34 Exit Layer	Prediction Engine
Theoretical FLOPs	100%	1.5%-13.6%	<0.001%	100%	1.5%-13.6%	<0.001%
Time on CPU (Intel i7-10700)	100%	8.2%-298.4%	<0.01%	100%	5.7%-224.4%	<0.01%
Time on SoC GPU (Jetson TX2)	100%	31.8%-125.5%	<0.8%	100%	50.8%-359.7%	<1.0%

where f_{mid} is the lowest frequency that allows inference to complete by T , and f_{high} is the platform's default high frequency. Thus, the energy consumption of this inference is

$$E = \int_0^{T_{L_0}} (CV_{high}^2 f_{high} + V_{high} N_{tr} I_{leakage} + P_{peripheral}) dt + \int_{T_{L_0}}^T (CV_{mid}^2 f_{mid} + V_{mid} N_{tr} I_{leakage} + P_{peripheral}) dt, \quad (16)$$

where T_{L_0} is the time of finishing the prediction at L_0 . As the selection of computing frequency is based on the predicted remaining workloads in the inference, the prediction engine can make the inference finish by time T based on correct predictions. Since the processor voltage and frequency are linear scales to the computing performance (inversely proportional to inference latency) but the square/cubic scale and linear scale to dynamic power and static power, the predictive exit will effectively reduce energy consumption.

This mechanism effectively minimizes the computation overhead caused by Exit Layers and Prediction Engines by activating the fewest rounds of exit modules. Furthermore, it is DVFS-friendly, as it ensures that the total computation time (inference latency) is guaranteed, allowing the inference to exit at the expected time once the processor voltage and frequency value are set. However, it is important to note that the predictions made by this method tend to be conservative under high network inference accuracy on a deep network. As there is only one decision making of our Low-Cost Prediction Engine, SPE cannot make the thorough distinction between those requiring largest number of layers to run and those can exit at the beginning. That is to say, for large prediction range (deep network) and big prediction variation (high accuracy requirement but still exist some inference need exit at the beginning) case, distribution of layers to exit by single prediction is more concentrated than it should be.

B. Multiple Prediction with Superior Efficiency

Complementary to the Single Prediction Exit (SPE), the Multiple Prediction Exit (MPE) mechanism allows for multiple early exit predictions, as illustrated in Fig 4. Unlike SPE, MPE does not force the inference to exit at a specific predicted point. Instead, at each potential exit layer, it evaluates the confidence level. If the threshold is not met, MPE activates the Prediction Engine to identify the next possible exit.

$$\zeta = \zeta_M(L_0, \beta) = \begin{cases} \zeta_S(L_0, \beta) = \text{PREDICT}\left(g_{\mathbf{W}_{L_0}}^{(L_0)}(y_{L_0}), \beta\right), & \alpha_W > \log \beta \\ \zeta_S(L_0, \beta) + \zeta_M(L_0 + \zeta_S(L_0, \beta), \beta), & \text{otherwise} \end{cases} \quad (17)$$

Because we do not know the final correct ζ at first, when applying DVFS, MPE choose to trust the predicted result of each prediction engine. That is to say, after i^{th} prediction, a new middle frequency $f_{mid,i}$ (and its corresponding voltage)

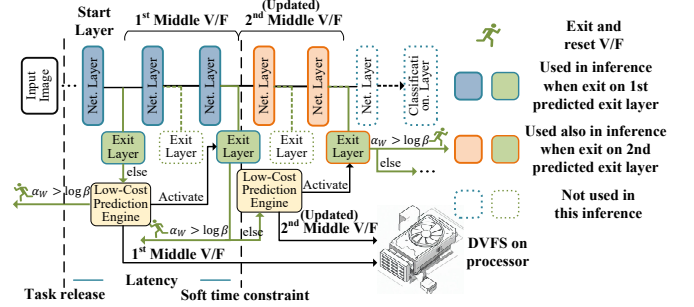


Fig. 4: Inference within Multiple Prediction Exit (MPE).

is set on the computation platform from current layer $L_{M,i}$ to the next predicted exit layer $L_{M,i+1}$, and it's value is based on the remaining time before deadline and the predicted layer to exit. Let $\zeta_{M,i}$ be the result of i^{th} prediction, i.e. $\zeta_{M,i} = L_{M,i+1} - L_{M,i}$, we have:

$$f_{mid,i} = \frac{\zeta_{M,i}}{L_{total} - L_{M,i}} f_{high}, \quad (18)$$

And the energy consumption in this inference would be:

$$E = \int_0^{T_{L_0}} (CV_{high}^2 f_{high} + V_{high} N_{tr} I_{leakage} + P_{peripheral}) dt + \sum_{i=1}^n \int_{T_{M,i}}^{T_{M,i+1}} (CV_{mid,i}^2 f_{mid,i} + V_{mid,i} N_{tr} I_{leakage} + P_{peripheral}) dt, \quad (19)$$

Where n is the prediction time of this inference and $T_{L_{M,i}}$ is the time of finishing the i^{th} prediction layer $L_{M,i}$ and its previous network layers.

Within MPE, for those able to exit at the early network layers, it's easier to exit at the correct place after the first prediction; while for those requiring more inference layer to run, MPE keep predicting until able to exit. Multiple prediction significantly improve the accuracy of the predictions. Although MPE activates more Exit Layers and Prediction Engines, the inference passes through fewer network layers (while maintaining the same inference accuracy) because better exit prediction utilize exit potential more effectively to achieve more efficient trade-off of computation cost and inference accuracy. However, it is worth noting that the DVFS mechanism is less efficient in this approach. Multiple voltage and frequency adjustments lead to inconsistency between the previous adjustments and later execution times, and each voltage and frequency adjustment incurs its own overhead in terms of adjustment time. Therefore, the multiple prediction MPE will have a more fine-grained prediction and thus saving more energy but cannot guarantee the inference latency.

VI. EVALUATION

A. Evaluation Setup

1) *Experiment Design:* We evaluate the proposed LEAP using classic CNN backbones such as **VGG19** [33] and

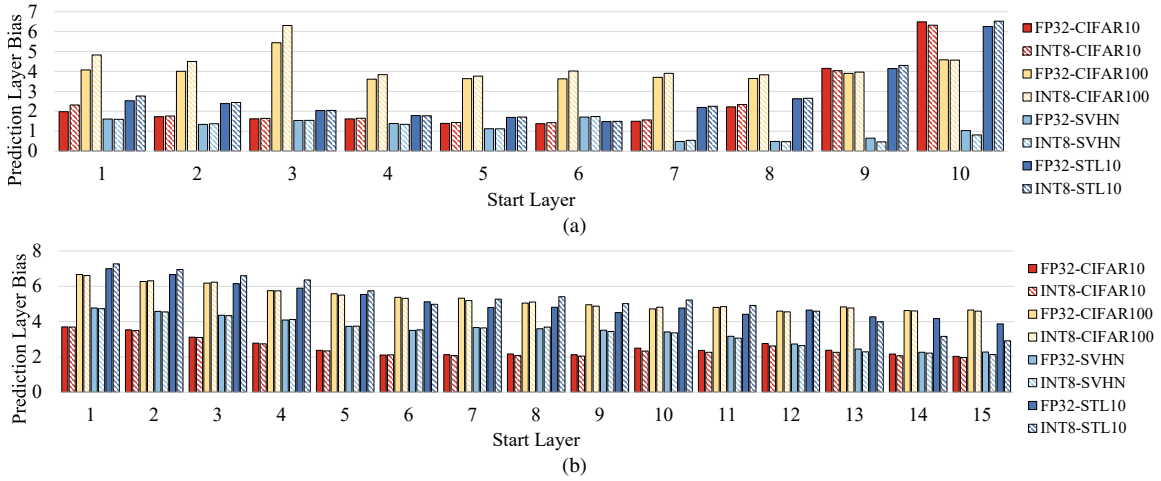


Fig. 5: Prediction deviation with hyperparameter L_0 for (a) VGG19 network and (b) ResNet34 network.

ResNet34 [34] on the commonly used CIFAR-10, CIFAR-100 [35], SVHN [36], and STL10 datasets [37]. Meanwhile, we also evaluate the applicability of LEAP on the novel vision transformer (ViT) models **DeiT-tiny** and **DeiT-small** [38] using the ImageNet [39] dataset. Both floating-point (FP32) and fixed-point quantization (INT8) operations are tested to consider applications with difference computing resource. The training follows the procedure in Section IV-A. For training on CIFAR-10, CIFAR-100, SVHN, and STL10, we train backbone and exit layer for 200 epochs each. For training on ImageNet, given the extensive scale of training set, we reduce the exit layer training to 50 epoch. As shown in Fig. 6, the energy consumption is measured on Intel i7-10700 desktop CPU and Jetson TX2 embedded SoC GPU with Tektronix MDO32 oscilloscope and TCP2020 current probe. The computation and energy overhead from exit layer and prediction engine are included in the tests.

We compare the proposed **LEAP** with different baselines:

- **Backbone Network (Backbone)**, directly inference without early exit mechanism.
- **Hierarchical Exit (Hier)**, proposed in [29], specifically designed for CNNs by placing exits at roughly $\frac{1}{4}$, $\frac{1}{2}$, $\frac{3}{4}$ of the network.
- **Placement Exit (Place)**, proposed in [13] and improved in [8] with tunable hyperparameters. It identifies the best place to attach an early exit by considering the benefit and extra computation and energy cost.
- **Oracle Exit (Oracle)** places an early exit layer after every neural network layer. It is used for evaluating the prediction and placement of exit layers.
- **Edge Exit (Edge)**, proposed in [10]. Although Edge incorporates DVFS and early exit similar to LEAP, its entropy-LUT-based exit prediction necessitates additional pre-training and LUT (Look-Up Table) distillation. While distilling the prediction model into an LUT reduces computational overhead, it compromises prediction accuracy (see Section VI-C for prediction performance comparison between Edge and LEAP). This makes Edge's exit mechanism quite conservative: like Oracle, Edge runs all the exit layer until the prediction point; after that, similar to LEAP's SPE, inference exits directly.

Among the above approaches, **Hier**, **Place**, **Edge** and **Oracle**

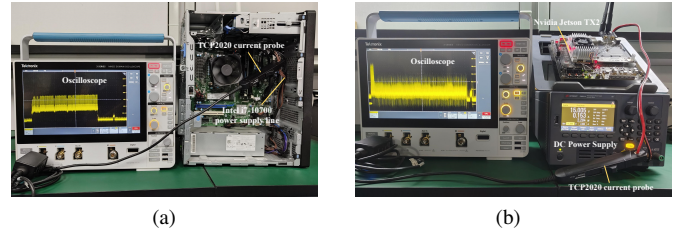


Fig. 6: Power/Energy measurement setup for (a) Intel i7-10700 (CPU) and (b) Jetson TX2 (GPU).

belong to static early exit placement approaches. On the contrary, the proposed **LEAP** is the dynamic early exit placement approach because it can dynamically adjust the exit placement for each input data.

2) **Exit Layer Parameter and Training Cost Overhead**: Prior to formal evaluation, we first assess the parameter and training overhead brought by attaching additional exit layers to the backbone. Please note that since the Place method determines exit placement by traversing all pre-trained exit layers before inference, it must train all exit layers like Edge, LEAP, and Oracle. While during inference, Place is similar to Hier in that only some exit layers are loaded into memory. As shown in Table III, for CNN, the additional parameters of exit layers are negligible. For ViT, using all exit layers for inference can bring up to 66% additional parameters. Since one of the key ideas of early exit technique is to trade off additional network structures for a significant reduction in inference computational cost, we assume that users have sufficient memory to accommodate these extra exit layers. Therefore, LEAP and baselines are evaluated without memory limits to maximize each approach's exit performance. Meanwhile, LEAP also supports manually restricting the exit layer number to reduce memory usage. For example, for DeiT-tiny, LEAP allows to modify the start layer L_0 to remove the first four exit layers. Furthermore, since LEAP's exit layers have the same topology, LEAP supports to fuse all exit classifiers. These optimizations reduce the additional exit parameters from 3.66M to 1.09M (66.3% to 19.7% of the backbone parameters) without significantly degrading performance.

In addition to increasing inference memory footprint, exit layers also requires additional training time cost. Table IV

TABLE III: Parameter number of backbone and exit layer.

Model	Backbone	Hier	Place	Full Load ¹
VGG19	20.04M	+0.11M(+0.5%)	+0.13M(+0.7%)	+0.63M(+3.1%)
ResNet34	21.28M	+0.09M(+0.4%)	+0.15M(+0.7%)	+0.19M(+0.9%)
DeiT-tiny	5.52M	+1.22M(+22.1%)	+1.36M(+24.7%)	≤+3.66M(+66.3%)
DeiT-Small	21.65M	+3.32M(+15.3%)	+5.85M(+27.0%)	≤+9.96M(+46.0%)

¹ Exit approaches that requires load all exit layers in memory during inference, including Edge, LEAP and Oracle. LEAP allows to manually set start layer and eliminate exit parameters before the start layer.

TABLE IV: GPU hours of backbone and exit layer training on NVIDIA GeForce RTX 3090.

CNN	VGG19			ResNet34		
	Training cost	Backbone	Hier	Full Train ¹	Backbone	Hier
CIFAR10		1.0	1.2	1.7	2.1	1.4
CIFAR100		1.0	1.2	1.6	2.0	1.2
SVHN		1.3	1.6	2.3	2.0	1.7
STL10		1.5	1.5	2.4	1.8	1.6
ViT	DeiT-tiny			DeiT-small		
	Training cost	Backbone	Hier	Full Train ¹	Backbone	Hier
ImageNet	180.0	14.1	36.5	250.0	17.4	48.7

¹ Exit approaches that requires training all exit layers, including Place, Edge, LEAP and Oracle.

presents the GPU hours (on NVIDIA GeForce RTX 3090) for training backbone and exit layers. On small datasets (CIFAR10, CIFAR100, SVHN, STL10), the training time difference between Full Train and Hier is not significant. Even if the training time of Full Train is added, the total GPU hours remain quite small. On ImageNet, since we reduced the exit training epoch to 50, the exit training GPU hour is much less than the backbone GPU hour. Therefore, the additional training cost of Full Train compared to Hier is relatively minor when considered alongside the backbone training time.

3) *Tunable Hyperparameters for Inference with Exit Layer:* The designs described in Section IV involve multiple tunable hyperparameters, which balance the inference accuracy, computation and energy. Table V provides a brief description of them. To fairly compare different exit methods, we fine-tune hyperparameters for maximum efficiency with minimal computation, while maintaining an acceptable accuracy loss. We ensure LEAP and other exit methods have no more than a 1.0% accuracy decrease compared to the backbone on the validation set except for Section VI-E, in which we tune the threshold hyperparameter β to evaluate the computation and energy cost of the model at different accuracy. In low-difficulty tasks, β is often close to 1, especially for SPE, which requires a strict β to ensure a sufficiently long prediction range. Therefore, we adjust it in the form of $1 - \beta$. Table VI presents an example of tuning β to trade-off inference accuracy and normalized computational cost. These data were used to form part of the SPE and MPE lines in Fig. 10.

B. Evaluation Metrics

1) *Metrics of Early Exit Prediction:* To quantitatively evaluate the exit point prediction performance, we first establish a baseline called Oracle Exit (Oracle) in a brute-force way. We add an exit layer after **every** network layer, regardless of its depth. Once the exit confidence score exceeds the threshold β , indicating the earliest correct exit point has been reached, the inference exits immediately. However, this brute-force approach introduces significant exit overhead compared to the saved computation cost, making it impractical for real use. In this section the Oracle Exit serves only as a theoretical baseline.

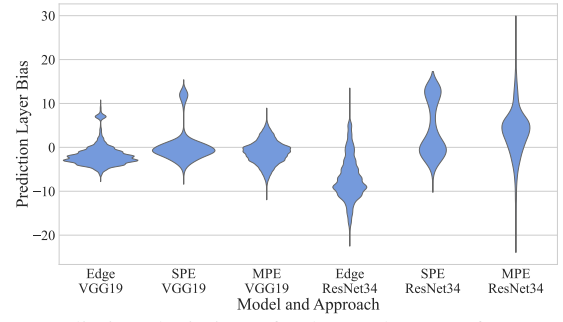


Fig. 7: Prediction deviation of Edge and LEAP for CIFAR10.

By running Oracle Exit, we record the actual exit points for each input as the ideal reference. The exit points predicted by LEAP or other early exit methods are then compared with these references. The deviation from them is used as the metric to evaluate early exit performance.

Since prediction performance is directly reflected to the overhead of the exit layer. We also evaluate the average number of exit layer executions for each exit method. The exit layer overhead and inference layer computation cost will be combined to form total computation cost in following end-to-end inference part.

2) *Metrics of End-to-end Inference:* For evaluation of end-to-end inference, we consider the following metrics.

- **Accuracy:** In early exit designs, our goal is to maximize efficiency by enabling the network to exit as early as possible while maintaining inference accuracy. Thus, we begin by evaluating the network's inference accuracy.
- **Computation Cost:** While keeping the inference accuracy of each exit method equal and fixed, we measured the average number of backbone layer execution and the average FLOPs (or INTOPs for quantized models) of each method on different models and datasets.
- **Energy and Latency Cost:** Keeping accuracy fixed, we measured the inference time and energy cost of each method on different processors. Then, we calculated the total energy consumption and the time distribution for processing 5000 data samples to demonstrate the energy and latency performance of each method.

C. Performance of Early Exit Prediction

To evaluate the accuracy of the prediction engine, we compare the predicted exit points and the oracle exit points obtained from the Oracle Exit. Note that the Oracle Exit does not ensure correct network inference, with or without early exits. It only identifies the earliest exit layer that meets the specified threshold. Fig. 5a and 5b present the one-time prediction error (the average bias from Oracle Exit) under FP32 and INT8 operations across different datasets and models when the exit and prediction start from different start layer L_0 . For VGG19, when L_0 is not large, the prediction engine can effectively limit the average bias within or around 2 layers for datasets like CIFAR10, SVHN, and STL10. Moving on to the deeper ResNet34, the prediction bias progressively reduces as the start layer L_0 going deeper. Generally, for SPE, a smaller L_0 allows more early exit opportunities due to a wider prediction range. However, deeper networks require a larger L_0 to maintain

TABLE V: Specific tunable hyperparameters.

Hyperparameter	Range	Description
β	$(0, 1)$	Confidence threshold. Control the trade-off of inference accuracy and computation cost.
L_0	$\mathbb{Z} \cap [0, L_{\text{total}}]$	Start layer. The index of layer to start applying early exit. For MPE, recommend to be 0; for SPE, recommend to be half of the depth. Increasing L_0 will enhance the prediction accuracy, but the potential for exit will decrease.
h	$\mathbb{R}^K$	Prediction engine convolution kernel. Each layer can have different learnable h , i.e. h_i . For MPE, recommend to be fixed $[1, \dots, 1, \dots, 1]$; for SPE, recommend to be sparse kernel $[x, \dots, x, 1, x, \dots, x]$ (small positive value x , the deeper the expected exit point, the smaller x is).
τ	$\mathbb{Z} \cap [0, L_{\text{total}}]$	Prediction upper bound. For SPE, recommend to be L_{total} ; for MPE, we keep it fixed as 8. Reducing τ will increase the MPE prediction accuracy, but it will also increase the execution times of exit layer and prediction.

TABLE VI: Tuning β to trade-off inference accuracy and normalized computation cost for CIFAR10 on VGG19.

	$1 - \beta$	0	1.7E-6	6.8E-6	3.0E-5	1.1E-4	8.7E-4	3.5E-3	7.0E-3	2.8E-2
SPE	Acc. (%)	91.98	91.08	90.74	90.50	90.30	89.58	89.00	88.46	88.02
	Comp. (%)	100.00	39.11	38.06	36.65	35.02	31.81	29.30	27.93	25.24
	$1 - \beta$	0	3.3E-2	5.3E-2	6.6E-2	7.9E-2	9.9E-2	1.3E-1	1.4E-1	1.5E-1
MPE	Acc. (%)	91.98	91.08	90.54	90.20	89.90	89.48	88.92	88.56	88.24
	Comp. (%)	100.00	29.22	26.90	25.89	24.99	23.57	22.03	20.99	20.70

prediction accuracy. For MPE, setting $L_0 = 0$ is sufficient, as its iterative exit mechanism allows inference to exit at the right place even if the initial prediction has high bias.

Fig. 7 more clearly illustrates the bias distribution between the exit prediction of Edge and LEAP and the actual exit points of the Oracle exit. As shown, Edge's predictions exhibit a notable pre-offset, especially on deeper networks like ResNet34. Since Edge's LUT predictor is built by extracting entropy from the validation set and using a small MLP for simple training and inference. Its data-driven prediction faces challenges in generalization and requires additional training, leading to relatively low prediction accuracy. Consequently, Edge fails to effectively reduce computational cost or energy consumption. In contrast, LEAP (SPE and MPE) predictions are mostly close to the actual Oracle Exit points. When comparing SPE and MPE, we find that SPE adopts a more conservative strategy than MPE. Specifically, SPE tends to predict deeper exit points than the Oracle baseline, as seen in the figure where SPE shows more cases with a positive prediction bias. This behavior results from SPE's requirement that the network must exit exactly at the predicted layer. To maintain inference accuracy, SPE often delays exiting until more layers are processed, making it more cautious. Comparing 19 layer VGG and 34 layer ResNet, we observe that both SPE and MPE become more conservative as network depth increases.

Table VII summarizes the average number of exit layers executed across different methods and datasets. As shown, the original backbone does not include any exit layers. In the shallower VGG19 network, the Hier and Place approaches use 1.56 and 1.11 exit layers on average for each inference. Regarding the SPE approach, the first executed exit layer located at L_0 . If the network does not exit at this point, it continues to the next predicted exit and exits from there. As a result, SPE uses an average of 1.78 layers per inference. In contrast, the MPE method performs repeated predictions, leading to more exit layers being executed than SPE. In the deeper ResNet34 network, the Hier and Place approaches use 1.63 and 1.84 exit layers on average, respectively higher than their performance on VGG19. However, SPE executes fewer exit layers as more samples exit at first exit layer. Thus, SPE executes the fewest exit layers on complex models ResNet34 and DeiT compared to other early exit approaches. Notably,

Edge and Oracle insert an exit layer after every network layer, which inevitably leads to more exit executions. This effect becomes more pronounced in deeper networks, as seen when moving from VGG19 to ResNet34.

D. Performance of End-to-End Inference

1) *End-to-End Inference Accuracy*: Table VIII shows the backbone accuracy across various datasets and the inference accuracy used to evaluate LEAP and other baseline methods. As mentioned before, by tuning the hyperparameters, the accuracy loss caused by LEAP and other early exit methods can be kept within acceptable ranges of 0.2% to 1.0%. To ensure a fair and meaningful comparison in the following evaluation, we adjust each method's hyperparameters so that all early exit approaches achieve similar end-to-end inference accuracy, with differences within 0.03%. This creates a level playing field for an unbiased assessment of their computational and energy efficiency. Additionally, we evaluate the trade-offs between inference accuracy and computation or energy cost for all early exit approaches at the end of this section.

2) *Computational Cost*: To quantify computational cost, we analyze the number of executed network layers for each approach across different datasets, as shown in Table IX. These values are normalized to the Backbone models. As expected, all exit approaches significantly reduce the number of executed layers. Compared to Hier and Place, the SPE has a similar layer count. While the MPE approach reduces the average number of executed layers by 12.1 (VGG19), 20.6 (ResNet34) and 1.7 (DeiT), outperforming other methods. Compared to Oracle baseline, MPE executes only 0.7 (VGG19), 0.6 (ResNet34) and 0.2 (DeiT) more layers on average. In contrast, Oracle executes 3.6 (VGG19), 7.7 (ResNet34) and 7.0 (DeiT) more exits than MPE, as shown in Table VII.

Table X presents the total computation cost of various approaches in terms of floating-point operations (FLOPs) and integer operations (INTOps), covering backbone layers, exit layers, and prediction cost. Compared to Hier and Place, the SPE achieves a similar computation cost. However, for the VGG19 model, MPE reduces the computation cost by an average of 5.0% compared to Hier and Place. Exit layer pre-placed approaches—Hier, Place, and Edge—perform well on shallow networks. However, their effectiveness diminishes as network depth increases. In contrast, on deeper networks like ResNet34, the MPE consistently achieves significant computation savings, ranging from 5.4% to 14.3% on average over other methods. The strength of MPE lies in its ability to dynamically adjust exit points during inference, enabling a balance between fine- and coarse-grained placement and offering a versatile solution across different network architectures.

TABLE VII: Average exit layer executed in each inference task.

CNN Approach	VGG19							ResNet34						
	Backbone	Hier	Place	Edge	SPE	MPE	Oracle	Backbone	Hier	Place	Edge	SPE	MPE	Oracle
FP32 CIFAR10	0.00	1.30	1.09	6.16	1.97	2.75	4.86	0.00	1.41	1.26	15.18	1.06	4.20	10.01
FP32 CIFAR100	0.00	1.94	1.05	8.33	2.00	3.42	7.93	0.00	2.27	3.17	23.60	1.32	4.33	19.50
FP32 SVHN	0.00	1.23	1.15	7.07	1.24	2.53	6.77	0.00	1.13	1.14	14.60	1.03	3.33	6.57
FP32 STL10	0.00	1.42	1.06	5.37	2.00	2.84	4.65	0.00	1.93	1.79	15.28	1.05	4.66	12.74
INT8 CIFAR10	0.00	1.29	1.05	6.37	1.96	3.04	5.32	0.00	1.39	1.26	15.96	1.06	4.21	9.66
INT8 CIFAR100	0.00	2.07	1.08	8.70	2.00	3.93	6.53	0.00	2.17	3.18	23.96	1.35	4.25	18.22
INT8 SVHN	0.00	1.22	1.24	7.07	1.85	1.95	6.97	0.00	1.12	1.09	14.83	1.03	3.16	6.10
INT8 STL10	0.00	2.03	1.14	5.12	1.25	3.39	9.58	0.00	1.65	1.82	14.72	1.05	4.52	11.74
CNN Average	0.00	1.56	1.11	6.77	1.78	2.98	6.58	0.00	1.63	1.84	17.27	1.12	4.08	11.82

ViT	DeiT-tiny							DeiT-small						
	Backbone	Hier	Place	Edge	SPE	MPE	Oracle	Backbone	Hier	Place	Edge	SPE	MPE	Oracle
FP32 ImageNet	0.00	3.48	2.64	10.60	1.71	2.96	10.24	0.00	3.20	3.41	9.62	1.48	2.49	9.41
INT8 ImageNet	0.00	3.48	2.54	10.58	1.71	2.99	10.21	0.00	3.04	3.29	9.67	1.49	2.42	9.01

TABLE VIII: Fixed accuracy for fair comparison.

CNN Approach	VGG19			ResNet34		
	Backbone	Early Exit	Accuracy	Backbone	Early Exit	Accuracy
FP32 CIFAR10	91.98%	91.08%±0.02%	93.64%	92.96%±0.02%		
FP32 CIFAR100	65.70%	65.00%±0.01%	71.54%	70.54%±0.01%		
FP32 SVHN	94.70%	93.88%±0.01%	95.28%	94.54%±0.01%		
FP32 STL10	87.08%	86.72%±0.01%	90.82%	90.56%±0.02%		
INT8 CIFAR10	90.68%	90.02%±0.01%	93.34%	92.86%±0.02%		
INT8 CIFAR100	60.80%	60.14%±0.01%	70.98%	70.18%±0.02%		
INT8 SVHN	94.82%	93.88%±0.01%	94.90%	94.18%±0.01%		
INT8 STL10	91.52%	90.72%±0.01%	92.02%	91.38%±0.01%		

ViT	DeiT-tiny			DeiT-small		
	Backbone	Early Exit	Accuracy	Backbone	Early Exit	Accuracy
FP32 ImageNet	72.80%	72.56%±0.03%	79.16%	78.90%±0.03%		
INT8 ImageNet	71.82%	71.10%±0.03%	79.28%	78.53%±0.03%		

TABLE IX: Average inference layer executed in each inference task.

CNN Approach	VGG19							ResNet34						
	Backbone	Hier	Place	Edge	SPE	MPE	Oracle	Backbone	Hier	Place	Edge	SPE	MPE	Oracle
FP32 CIFAR10	19.00	6.84	6.67	7.16	7.19	5.21	4.40	34.00	13.54	16.54	16.18	14.31	11.77	11.03
FP32 CIFAR100	19.00	9.97	10.05	9.33	9.49	8.37	7.21	34.00	21.44	21.91	24.60	25.01	21.02	20.60
FP32 SVHN	19.00	6.46	6.90	8.07	6.84	6.32	6.13	34.00	11.06	16.29	15.60	12.14	8.19	7.57
FP32 STL10	19.00	7.41	6.56	6.38	7.71	5.01	4.21	34.00	18.43	19.70	16.28	19.57	13.97	13.80
INT8 CIFAR10	19.00	6.78	6.50	7.37	7.10	5.74	4.82	34.00	13.37	16.53	16.96	15.17	11.77	10.67
INT8 CIFAR100	19.00	10.68	10.14	9.70	9.39	9.25	7.79	34.00	20.41	21.93	24.96	24.24	19.86	19.28
INT8 SVHN	19.00	6.40	7.00	8.07	7.60	6.36	6.31	34.00	10.96	16.18	15.83	12.00	7.58	7.10
INT8 STL10	19.00	10.61	11.05	6.12	11.89	9.29	8.79	34.00	15.70	19.78	15.72	18.65	13.48	12.80
CNN Average	19.00	8.15	8.11	7.78	8.40	6.94	6.21	34.00	15.61	18.61	18.27	17.64	13.45	12.86

ViT	DeiT-tiny							DeiT-small						
	Backbone	Hier	Place	Edge	SPE	MPE	Oracle	Backbone	Hier	Place	Edge	SPE	MPE	Oracle
FP32 ImageNet	12.00	10.43	10.64	10.60	10.96	10.36	10.24	12.00	9.61	9.41	9.62	10.26	9.57	9.41
INT8 ImageNet	12.00	10.45	10.54	10.58	10.94	10.28	10.21	12.00	9.12	9.29	9.67	10.26	9.38	9.01

TABLE X: Computation cost (floating point operations/integer operations) in each inference task.

CNN Approach	VGG19							ResNet34						
	Backbone	Hier	Place	Edge	SPE	MPE	Oracle	Backbone	Hier	Place	Edge	SPE	MPE	Oracle
FP32 CIFAR10	100.00%	36.84%	35.80%	47.04%	39.11%	29.22%		100.00%	40.40%	49.15%	53.67%	42.50%	36.29%	
FP32 CIFAR100	100.00%	53.75%	53.59%	61.56%	51.24%	46.27%		100.00%	63.96%	65.69%	81.79%	74.09%	63.56%	
FP32 SVHN	100.00%	34.78%	37.04%	53.14%	36.82%	34.93%		100.00%	32.99%	48.36%	51.73%	36.12%	25.42%	
FP32 STL10	100.00%	39.93%	35.23%	41.80%	41.86%	28.21%		100.00%	54.97%	58.64%	53.98%	57.97%	42.94%	
INT8 CIFAR10	100.00%	36.52%	34.90%	48.45%	38.62%	32.19%		100.00%	39.87%	49.12%	56.27%	45.05%	36.30%	
INT8 CIFAR100	100.00%	57.57%	54.07%	64.02%	50.73%	51.23%		100.00%	60.89%	65.78%	83.00%	71.83%	60.12%	
INT8 SVHN	100.00%	34.49%	37.65%	53.15%	41.20%	34.72%		100.00%	32.67%	48.02%	52.50%	35.70%	23.56%	
INT8 STL10	100.00%	57.17%	58.91%	40.09%	63.39%	51.10%		100.00%	46.84%	58.89%	52.12%	55.29%	41.46%	
CNN Average	100.00%	43.88%	43.40%	51.16%	45.37%	38.48%		100.00%	46.57%	55.46%	60.63%	52.32%	41.21%	

ViT	DeiT-tiny							DeiT-small						
	Backbone	Hier	Place	Edge	SPE	MPE	Oracle	Backbone	Hier	Place	Edge	SPE	MPE	Oracle
FP32 ImageNet	100.00%	88.59%	89.79%	94.60%	91.82%	87.65%		100.00%	81.75%	80.24%	86.17%	85.93%	80.15%	
INT8 ImageNet	100.00%	88.74%	88.86%	94.37%	91.69%	87.07%		100.00%	78.01%	79.17%	86.66%	85.93%	77.35%	

3) *Energy Consumption and Inference Latency*: Finally, we evaluate the power and energy benefits of LEAP. As mentioned before, LEAP and Edge have their own DVFS optimization accompanied with their exit mechanism. To ensure a fair comparison, for the passive methods Hier and Place, the processor voltage and frequency are set to the lowest values after inference exits and before the next round begins. Table XII and XI summarize the energy consumption of different approaches on the Intel i7-10700 and Jetson TX2. As expected, all early exit approaches achieve significant energy savings compared to the backbone model. As previously analyzed, SPE performs well on moderately deep networks. On VGG19, SPE further reduces energy consumption by 22.6% to 26.9% on the i7-10700 platform and up to 20.2% on the Jetson TX2 platform, compared to the best cases of Hier and Place, by predicting exit points and adjusting computation configurations during inference. The energy savings on the TX2 are less than on i7-10700 because the power of the entire development kit is measured, which includes a large idle power, thereby diluting the energy savings achieved through DVFS.

Although SPE performs poorly on deep networks, MPE achieves significant power savings even on the TX2 platform, while maintaining similar performance on shallow networks. On VGG19, MPE further reduces energy consumption by 7.6%

TABLE XI: Energy consumption on Intel I7-10700 (Joule).

CNN Approach	VGG19							ResNet34						
	Backbone	Hier	Place	Edge	SPE	MPE	Oracle	Backbone	Hier	Place	Edge	SPE	MPE	Oracle
CIFAR10	3899	1179	1099	896	638	602		5334	2456	2939	2534	2557	1365	
CIFAR100	3901	1802	1536	1024	1139	867		5730	3900	3991	3911	3712	1896	
SVHN	3926	1053	1145	888	807	695		5684	2104	2912	2675	2201	985	
STL10	3894	1312	1034	742	800	653		5735	3527	3591	2591	3726	1513	

ViT	DeiT-tiny							DeiT-small						
	Backbone	Hier	Place	Edge	SPE	MPE	Oracle	Backbone	Hier	Place	Edge	SPE	MPE	Oracle
ImageNet	8153	8064	7897	5439	5028	4448		14161	13030	12010	9650	10018	8979	

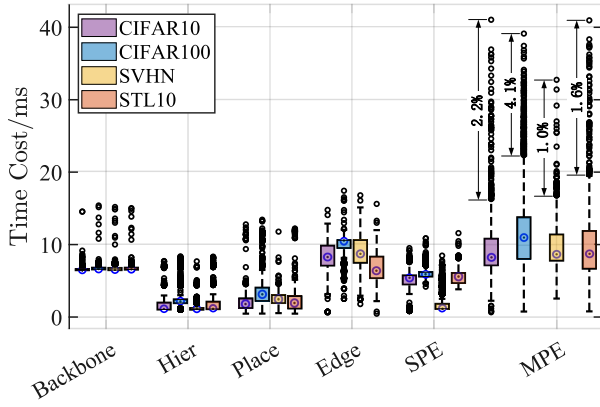
TABLE XII: Energy consumption on Jetson TX2 (Joule).

CNN Approach	VGG19							ResNet34						
	Backbone	Hier	Place	Edge	SPE	MPE	Oracle	Backbone	Hier	Place	Edge	SPE	MPE	Oracle
CIFAR10	894	308	283	337	243	246		1013	446	478	588	548	392	
CIFAR100	736	421	374	424	389	340		971	689	712	915	879	574	
SVHN	701	283	297	372	226	241		957	384	457	554	481	316	
STL10	784	335	290	302	260	261		1226	591	614	731	728	452	

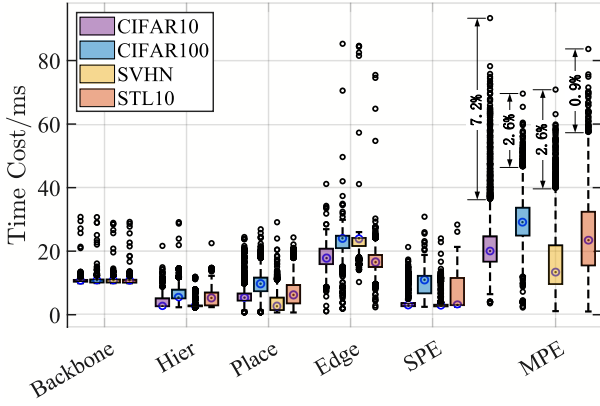
ViT	DeiT-tiny							DeiT-small						
	Backbone	Hier	Place	Edge	SPE	MPE	Oracle	Backbone	Hier	Place	Edge	SPE	MPE	Oracle
ImageNet	1174	1012	956	979	1000	916		2527	2366	2205	2346	2283	2114	

to 23.9% on i7-10700 and shows comparable results on TX2 compared to SPE. On the deeper ResNet34, SPE becomes less effective, while MPE reduces energy consumption by 41.6% to 57.1% on i7-10700 and 12.2% to 23.4% on TX2 compared to the best cases of Hier, Place and Edge. Even on DeiT, where the opportunity to exit is very limited, MPE remains the most energy-efficient among all the methods. This is because MPE dynamically adjusts exit points during inference, leading to more accurate predictions and fewer executed layers.

Fig. 8 and Fig. 9 summarize the inference latencies of



(a)



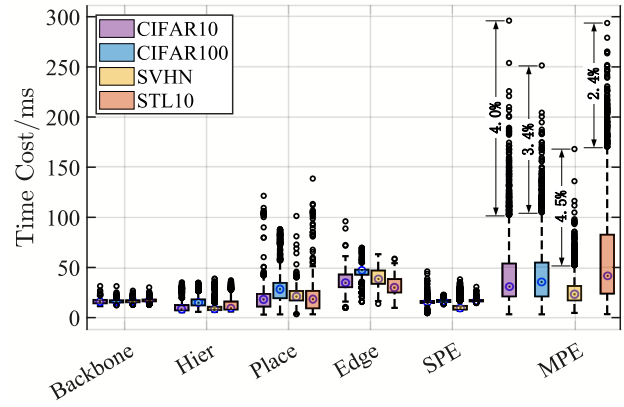
(b)

Fig. 8: Statistical results for latency of different approaches on Intel i7-10700. (a) VGG19 network. (b) ResNet34 network.

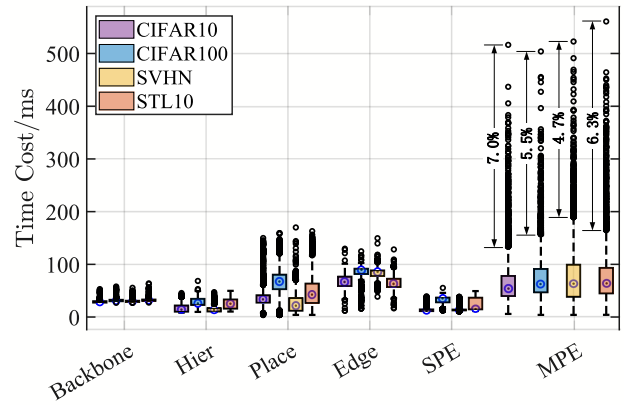
different models on various computing platforms. Since SPE only performs one prediction and force inference to exit at predicted point, its execution pattern allows for precise voltage and frequency settings in advance. As a result, inference latency is predictable and guaranteed. In contrast, MPE considers multiple predicted points as potential exits during inference, leading to an approximate execution pattern after voltage and frequency settings are made. This makes latency uncertain. However, predictions are generally accurate, and the network typically exits at the first predicted point, with only a few cases requiring multiple trials. Statistical analysis shows that only a small percentage (0.9% to 7.2%) of MPE inferences may exceed the worst-case execution time observed in other early exit approaches. Although Edge and SPE both make only one prediction, Edge does not always exit at the predicted point, resulting in significantly higher latency than SPE despite similar energy consumption. In contrast, MPE achieves notably lower energy consumption while maintaining an average latency comparable to Edge.

E. Computation, Energy and Accuracy Trade-offs

Choosing the confidence threshold for early exit (ie, β), the trade-off of inference accuracy, computation cost, and energy consumption are balanced. Fig. 10 and Fig. 11 illustrate the trade-off among these metrics, focusing on the VGG19 model with the CIFAR10 dataset on Intel i7-10700 processor. Similar trends are observed across other datasets, the ResNet34 model,



(a)



(b)

Fig. 9: Statistical results for latency of different approaches on Jetson TX2. (a) VGG19 network. (b) ResNet34 network.

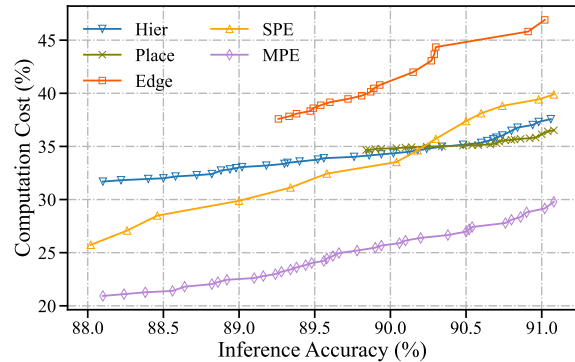


Fig. 10: Trade-off between computation cost and accuracy in different early exit method for CIFAR10 on VGG19.

and the Jetson TX2 platform. According to the figures, the time-guaranteed SPE achieves a lower computation cost than Place, Hier and Edge in low accuracy level. However, under high accuracy requirements, its single prediction mechanism must predict farther and tolerate greater prediction bias, which incurs up to 2.5%-3.5% higher cost. In contrast, MPE achieves 7.5%-10.0% more computation savings at all accuracy levels. Regarding energy consumption, SPE and MPE consistently outperform state-of-the-art early exit methods under various accuracy requirements by accurate predicting and proactively adjusting processor voltage and frequency during inference.

To summarize the strengths of SPE and MPE, the proposed SPE offers guaranteed low inference latency with moderate

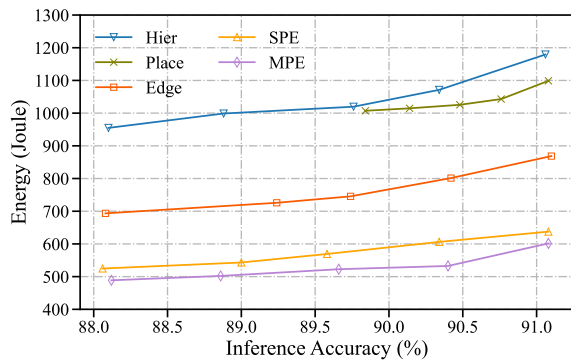


Fig. 11: Trade-off between energy (Intel i7-10700) and accuracy in different early exit method for CIFAR10 on VGG19.

computation and energy. Having predictable latency, SPE is suitable for real-time applications such as self-driving cars, robotics, and industrial automation. Meanwhile, MPE occasionally has longer latency but achieves significantly lower computation and energy costs. Its performance improves as networks become deeper, making MPE well-suited for QoS-aware applications like image and video processing.

VII. CONCLUSION

The proposed LEAP is an innovative and efficient inference technique that accurately predicts the network's exit point, leading to significant gains in computation and energy efficiency. By intelligently activating the exit layer at the expected exiting point, LEAP effectively reduces computation costs by eliminating the need to execute every pre-placed exit layer. Notably, LEAP achieves a remarkable reduction in energy consumption by selecting optimal computing configurations during inference. It streamlines the inference process, thereby contributing to substantial energy savings. Although LEAP is currently designed and evaluated for CNNs and ViTs, it can be seamlessly integrated as an add-on module into existing networks without modifying the backbone model. This makes LEAP versatile and applicable to various architectures, including GNN, RNN and LLMs. We plan to explore its potential in these areas in future work.

REFERENCES

- [1] D. Jesse *et al.*, "Measuring the carbon intensity of ai in cloud instances," in *Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency*, 2022, pp. 1877–1894.
- [2] C.-J. o. Wu, "Sustainable ai: Environmental implications, challenges and opportunities," *Proceedings of Machine Learning and Systems*, vol. 4, pp. 795–813, 2022.
- [3] D. Stutz, N. Chandramoorthy, M. Hein, and B. Schiele, "Random and adversarial bit error robustness: Energy-efficient and secure dnn accelerators," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, pp. 3632–3647, 2022.
- [4] S. Teerapittayanon, B. McDanel, and H.-T. Kung, "Branchynet: Fast inference via early exiting from deep neural networks," in *2016 23rd International Conference on Pattern Recognition (ICPR)*. IEEE, 2016.
- [5] M. Figurnov *et al.*, "Spatially adaptive computation time for residual networks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017, pp. 1039–1048.
- [6] Y. Han, G. Huang, S. Song, L. Yang, H. Wang, and Y. Wang, "Dynamic neural networks: A survey," *IEEE Transactions on*

- Pattern Analysis and Machine Intelligence*, vol. 44, no. 11, pp. 7436–7456, 2021.
- [7] S. Scardapane, M. Scarpiniti, E. Baccarelli, and A. Uncini, "Why should we add early exits to neural networks?" *Cognitive Computation*, vol. 12, no. 5, pp. 954–966, 2020.
- [8] E. Baccarelli, S. Scardapane, M. Scarpiniti, A. Momenzadeh, and A. Uncini, "Optimized training and scalable implementation of conditional deep neural networks with early exits for fog-supported iot applications," *Information Sciences*, vol. 521, pp. 107–143, 2020.
- [9] B. Jacob, S. W. Ng, and D. T. Wang, "Overview: On memory systems and their design," in *Memory Systems*, B. Jacob, S. W. Ng, and D. T. Wang, Eds. San Francisco: Morgan Kaufmann, 2008, pp. 1–54.
- [10] T. Tambe *et al.*, "Edgebert: Sentence-level energy optimizations for latency-aware multi-task nlp inference," in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021, pp. 830–844.
- [11] M. Wang, J. Mo, J. Lin, Z. Wang, and L. Du, "Dynexit: A dynamic early-exit strategy for deep residual networks," in *IEEE International Workshop on Signal Processing Systems (SiPS)*, 2019, pp. 178–183.
- [12] V. Bonato and C.-S. Bouganis, "Class-specific early exit design methodology for convolutional neural networks," *Applied Soft Computing*, vol. 107, 2021.
- [13] P. Panda, A. Sengupta, and K. Roy, "Energy-efficient and improved image recognition with conditional deep learning," *ACM Journal on Emerging Technologies in Computing Systems (JETC)*, vol. 13, no. 3, pp. 1–21, 2017.
- [14] M. Wołczyk, B. Wójcik, K. Bałazy, I. T. Podolak, J. Tabor, M. Śmieja, and T. Trzcinski, "Zero time waste: Recycling predictions in early exit neural networks," in *Advances in Neural Information Processing Systems*, vol. 34, 2021, pp. 2516–2528.
- [15] L. Zhang, C. Bao, and K. Ma, "Self-distillation: Towards efficient and compact neural networks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 8, pp. 4388–4403, 2021.
- [16] Z. Jie, P. Sun, X. Li, J. Feng, and W. Liu, "Anytime recognition with routing convolutional networks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 43, no. 6, pp. 1875–1886, 2019.
- [17] X. Li*, Y. Shen*, A. Zou, and Y. Ma, "Eenet: Energy efficient neural networks with run-time power management," in *Proceedings of the Design Automation Conference (DAC)*, 2023.
- [18] S. M. Nabavinejad, S. Reda, and M. Ebrahimi, "Coordinated batching and dvfs for dnn inference on gpu accelerators," *IEEE transactions on parallel and distributed systems*, vol. 33, no. 10, pp. 2496–2508, 2022.
- [19] M. Odema, N. Rashid, and M. A. A. Faruque, "Eexas: Early-exit neural architecture search solutions for low-power wearable devices," in *2021 IEEE/ACM International Symposium on Low Power Electronics and Design*, 2021.
- [20] E. Li, L. Zeng, Z. Zhou, and X. Chen, "Edge ai: On-demand accelerating deep neural network inference via edge computing," *IEEE Transactions on Wireless Communications*, vol. 19, no. 1, pp. 447–457, 2020.
- [21] L. Zeng, E. Li, Z. Zhou, and X. Chen, "Boomerang: On-demand cooperative deep neural network inference for edge intelligence on the industrial internet of things," *IEEE Network*, vol. 33, no. 5, pp. 96–103, 2019.
- [22] X. Xu, Q. Lu, T. Wang, Y. Hu, C. Zhuo, J. Liu, and Y. Shi, "Efficient hardware implementation of cellular neural networks with incremental quantization and early exit," *J. Emerg. Technol. Comput. Syst.*, vol. 14, no. 4, dec 2018.
- [23] Y. Bengio, P. Lamblin, D. Popovici, and H. Larochelle, "Greedy layer-wise training of deep networks," in *Advances in Neural Information Processing Systems*, vol. 19. MIT Press, 2006.
- [24] E. Belilovsky, M. Eickenberg, and E. Oyallon, "Greedy layerwise learning can scale to imagenet," in *Proceedings of the 36th*

International Conference on Machine Learning. PMLR, May 2019, pp. 583–593.

- [25] Y. You, T. Chen, Z. Wang, and Y. Shen, “L2-gcn: Layer-wise and learned efficient training of graph convolutional networks,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2020, pp. 2127–2135.
- [26] H. Aly, A. K. Al-Ali, and P. N. Suganthan, “Boosted multilayer feedforward neural network with multiple output layers,” *Pattern Recognition*, vol. 156, p. 110740, Dec. 2024.
- [27] A. Karimi, A. Kalhor, and M. Sadeghi Tabrizi, “Forward layer-wise learning of convolutional neural networks through separation index maximizing,” *Scientific Reports*, vol. 14, no. 1, p. 8576, Apr. 2024.
- [28] O. Pina and V. Vilaplana, “Layer-wise training of graph neural networks with self-supervised learning,” *IEEE Transactions on Neural Networks and Learning Systems*, pp. 1–11, 2025.
- [29] N. Passalis, J. Raitoharju, A. Tefas, and M. Gabbouj, “Efficient adaptive inference for deep convolutional neural networks using hierarchical early exits,” *Pattern Recognition*, vol. 105, p. 107346, 2020.
- [30] S. Teerapittayanon, B. McDanel, and H. Kung, “Distributed deep neural networks over the cloud, the edge and end devices,” in *2017 IEEE 37th International Conference on Distributed Computing Systems (ICDCS)*, 2017, pp. 328–339.
- [31] Y. Kaya, S. Hong, and T. Dumitras, “Shallow-deep networks: Understanding and mitigating network overthinking,” in *International conference on machine learning*. PMLR, 2019.
- [32] G. Kim and J. Park, “Low cost early exit decision unit design for cnn accelerator,” in *2020 International SoC Design Conference (ISOC)*. IEEE, 2020, pp. 127–128.
- [33] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” *arXiv preprint arXiv:1409.1556*, 2014.
- [34] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016.
- [35] A. Krizhevsky, G. Hinton *et al.*, “Learning multiple layers of features from tiny images,” *Technical report, University of Toronto*, 2009.
- [36] Y. Netzer, T. Wang, A. Coates, A. Bissacco, B. Wu, and A. Y. Ng, “Reading digits in natural images with unsupervised feature learning,” in *NIPS Workshop on Deep Learning and Unsupervised Feature Learning*, 2011.
- [37] A. Coates, A. Ng, and H. Lee, “An analysis of single-layer networks in unsupervised feature learning,” in *Proceedings of the fourteenth international conference on artificial intelligence and statistics*. JMLR Workshop and Conference Proceedings, 2011, pp. 215–223.
- [38] H. Touvron *et al.*, “Training data-efficient image transformers & distillation through attention,” in *International Conference on Machine Learning*, vol. 139, July 2021, pp. 10 347–10 357.
- [39] O. Russakovsky *et al.*, “ImageNet Large Scale Visual Recognition Challenge,” *International Journal of Computer Vision (IJCV)*, vol. 115, no. 3, pp. 211–252, 2015.



Yingtao Shen (Student Member, IEEE) received his B.S. degree from Shanghai Jiao Tong University in 2022. he is currently pursuing the Ph.D. degree at Efficient and High-Performance Computing Laboratory (EHPCL) of the University of Michigan - Shanghai Jiao Tong University Joint Institute (UM-SJTU JI). His research interests include neural network dynamic inference, model compression, and software and hardware co-design.



Xiangjie Li is a graduate student in Electrical Engineering and Information Systems Department at the University of Tokyo. He received his B.S. degree from Shanghai Jiao Tong University in 2022, during which he studied for computation and power efficient AI. He now belongs to 3D Vision Laboratory, exploring 3D computer vision and robotics.



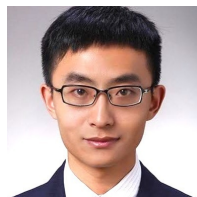
communication, and control in cyber-physical systems.

Yehan Ma (Member, IEEE) received the B.S. and M.S. degrees from Harbin Institute of Technology in 2013 and 2015, respectively, and the Ph.D. degree in computer science from Washington University in St. Louis in 2020. She is currently an Associate Professor with the School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University. Her research focuses on the Industrial Internet of Things, cyber-physical systems, and edge computing. She broadly investigated techniques and solutions for holistic managements of computation,



his work has also been nominated for Best Paper Award at DATE 2019 and DAC 2022.

Weidong Cao (Member, IEEE) received the Ph.D. degree from Washington University in St. Louis, St. Louis, MO, USA, in 2021. He is an Assistant Professor with the Department of Electrical and Computer Engineering, The George Washington University, Washington, DC, USA. Before that, he was a Principal Engineer with TSMC Corporate Research, San Jose, CA, USA. His research interests focus on VLSI design, computer architecture, EDA, and quantum computing. Dr. Cao is the recipient of IEEE/ACM ISLPED Best Paper Award in 2022 and



Jie Zhao is a senior applied scientist at Microsoft. His research focuses on natural language processing and artificial intelligence in mobile and edge computing. Dr. Jie Zhao received his Ph.D. degree in Computer Science and Engineering from the Ohio State University in 2020. His work has been published at top-tier conferences and Journals, such as EMNLP, SIGKDD, IJCAI, INFOCOM, Signal Processing, etc.



Aided Design of Integrated Circuits and Systems, ACM Transactions on Architecture and Code Optimization, and RTAS. His research focuses on computer architecture and embedded systems.

An Zou (Member, IEEE) received the B.S. and M.S. degrees from Harbin Institute of Technology, Harbin, China, in 2013 and 2015, respectively, and the Ph.D. degree in electrical engineering from Washington University, St. Louis, MO, USA, in 2021. He is an Associate Professor with the University of Michigan-Shanghai Jiao Tong University Joint Institute, Shanghai Jiao Tong University, Shanghai, China. His work has been extensively published and recognized at top-tier conferences and journals, including Micro, DAC, ICCAD, IEEE Transactions on Computer-